```
using BambooEnterprise.Data;
using BambooEnterprise.Models;
using BambooEnterprise.Testing;
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Net.Http;
using System.Text.Json;
using System.Threading.Tasks;

namespace BambooEnterprise
{
    // Concrete class to fix dynamic lambda dispatch errors
    public class EnrichedInventoryItem
    {
        public InventoryBatch inv { get; set; }
        public string SpeciesName { get; set; }
    }

    class Program
    {
        static List<BambooSpecies> SpeciesDb = new();
        static List<InventoryBatch> InventoryDb = new();
        static List<PerformanceLog> GrowthDb = new();
        static List<OperationalExpense> ExpenseDb = new();
        private static readonly HttpClient _httpClient = new();
        private static System.Timers.Timer _autoSaveTimer;

        static void Main(string[] args)
        {
            try
            {
                SetupAutoSave();
                string invPath = Path.Combine
                  (AppDomain.CurrentDomain.BaseDirectory, "Data",
                  "inventory.json");
                if (File.Exists(invPath))
                {
                    Console.WriteLine("=== SYSTEM STARTUP ===");
                    Console.Write("Database detected. Load existing data?
                      (Y/N): ");
                    if (Console.ReadLine()?.ToUpper() == "Y")
                    {
                        LoadAllData();
                        Console.WriteLine("Data loaded successfully.");
                    }
                    else
```

```
                    {
                        SeedSampleData();
                        Console.WriteLine("System initialized with Sample Data.");
                    }
                }
                else
                {
                    SeedSampleData();
                    Console.WriteLine("No database found. Initialized with Sample Data.");
                }
                Console.WriteLine("Press any key to enter Main Menu...");
                Console.ReadKey();
                MainMenu();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"Startup Error: {ex.Message}");
                Console.ReadKey();
            }
        }

        #region System Infrastructure
        private static void SetupAutoSave()
        {
            _autoSaveTimer = new System.Timers.Timer(300000);
            _autoSaveTimer.Elapsed += (sender, e) => PerformAutoSave();
            _autoSaveTimer.AutoReset = true;
            _autoSaveTimer.Enabled = true;
        }

        private static void PerformAutoSave()
        {
            PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
            PersistenceManager.SaveToJson(SpeciesDb, "species.json");
            PersistenceManager.SaveToJson(GrowthDb, "logs.json");
            PersistenceManager.SaveToJson(ExpenseDb, "expenses.json");
        }

        static void LoadAllData()
        {
            SpeciesDb = PersistenceManager.LoadFromJson<BambooSpecies>("species.json");
            InventoryDb = PersistenceManager.LoadFromJson<InventoryBatch>("inventory.json");
            GrowthDb = PersistenceManager.LoadFromJson<PerformanceLog>("logs.json");
            ExpenseDb =
```

```
                PersistenceManager.LoadFromJson<OperationalExpense>
                ("expenses.json");
        }

        static void MainMenu()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("=== BAMBOO INFORMATION MANAGEMENT
                  SYSTEM ===");
                ShowActiveAlerts();
                Console.WriteLine("\n1. Manage Biological Profiles
                  (DNA)");
                Console.WriteLine("2. Track Inventory & Acquisition");
                Console.WriteLine("3. Log Growth & Harvests");
                Console.WriteLine("4. Operational Expenses (OpEx)");
                Console.WriteLine("5. Financial Reports (P&L)");
                Console.WriteLine("6. Database Management (Backup/
                  Restore)");
                Console.WriteLine("Q. Quit Application");
                Console.Write("\nSelect: ");
                string choice = Console.ReadLine()?.ToUpper();
                if (choice == "Q") return;
                try { HandleChoice(choice); }
                catch (Exception ex) when (ex.Message == "GoBack")
                  { continue; }
            }
        }

        static void HandleChoice(string choice)
        {
            switch (choice)
            {
                case "1": ManageSpeciesFlow(); break;
                case "2": ManageInventoryFlow(); break;
                case "3": ManageGrowthFlow(); break;
                case "4": ManageExpenseFlow(); break;
                case "5":
                    PersistenceManager.GeneratePLReport(InventoryDb,
                      GrowthDb, ExpenseDb);
                    PersistenceManager.ExportToExcel(InventoryDb,
                      "Inventory_Report");
                    Console.WriteLine("\nReports generated. Press any
                      key...");
                    Console.ReadKey(); break;
                case "6":
                    Console.Clear();
                    Console.WriteLine("--- 6. DATABASE MANAGEMENT ---");
```

```
                    Console.WriteLine("1. Re-Seed Sample Data | 2. Restore from Backup | B. Back");
                    string dbChoice = Console.ReadLine()?.ToUpper();
                    if (dbChoice == "1") { SeedSampleData(); Console.WriteLine("Data Re-Seeded."); Console.ReadKey(); }
                    else if (dbChoice == "2") RestoreFlow();
                    break;
                }
            }
            #endregion

            #region Module 1: Biological Profiles
            static void ManageSpeciesFlow()
            {
                while (true)
                {
                    Console.Clear();
                    Console.WriteLine("--- BIOLOGICAL PROFILES ---");
                    Console.WriteLine("S. Search | V. View All | B. Back");
                    string cmd = Console.ReadLine()?.ToUpper();
                    if (cmd == "B") return;

                    IEnumerable<BambooSpecies> displayList = SpeciesDb;
                    if (cmd == "S")
                    {
                        Console.Write("Search term: ");
                        string filter = Console.ReadLine()?.ToLower();
                        displayList = SpeciesDb.Where(s => ($"{s.Genus} {s.Species} {s.Cultivar}").ToLower().Contains(filter));
                    }

                    int globalCounter = 1;
                    foreach (var s in displayList)
                    {
                        if ((globalCounter - 1) % 10 == 0)
                            Console.WriteLine("\n{0,-4} {1,-12} {2,-42} {3,-6} {4,-6} {5}", "Num", "UID", "Scientific Name \"Cultivar\"", "Hardy", "MaxHt", "Research Notes");

                        string name = $"{s.Genus} {s.Species} \"{s.Cultivar} \"".Trim();
                        Console.WriteLine("{0,-4} {1,-12} {2,-42} {3,-6} {4,-6} {5}", globalCounter, s.PlantUID, name, s.HardinessLow + "C", s.HeightHigh + "m", s.SverigeSpecifikt);
                        globalCounter++;
```

```
                }
                Console.WriteLine("\nPress any key..."); Console.ReadKey();
                if (cmd != "S" && cmd != "V") break;
            }
        }
        #endregion

        #region Module 2: Inventory Management
        static void ManageInventoryFlow()
        {
            while (true)
            {
                try
                {
                    Console.Clear();
                    Console.WriteLine("--- 2. TRACK INVENTORY ---");
                    Console.WriteLine("1. View All | 2. Summary View | 3. Add | 4. Update | 5. Delete | B. Back");

                    string cmd = Console.ReadLine()?.ToUpper();
                    if (cmd == "B") return; // This 'return' goes to MainMenu correctly

                    if (cmd == "1") { ViewInventory(); Console.WriteLine("\nPress any key..."); Console.ReadKey(); }
                    else if (cmd == "2") ViewInventorySummary();
                    else if (cmd == "3") AddInventoryFlow().GetAwaiter().GetResult();
                    else if (cmd == "4") UpdateInventoryFlow().GetAwaiter().GetResult();
                    else if (cmd == "5") DeleteInventoryFlow();
                }
                catch (Exception ex) when (ex.Message == "GoBack")
                {
                    // This catches the 'B' from DeleteInventoryFlow or ViewInventorySummary
                    // and simply 'continues' this loop, staying in the Inventory menu.
                    continue;
                }
            }
        }


        static async Task AddInventoryFlow()
        {
            while (true)
            {
```

```
                try
                {
                    Console.Clear();
                    Console.WriteLine("--- ADD NEW INVENTORY ---");
                    // Requirement: Clarify YYYYMMDD format
                    DateTime date = SafeRequestDate("Purchase Date (YYYYMMDD or YYYY-MM-DD)");
                    string nursery = RequestInput("Nursery Name", true);

                    string selectedUid = SelectPlantUID();

                    string nPart = nursery.Length >= 3 ? nursery.Substring(0, 3).ToUpper() : nursery.ToUpper().PadRight(3, 'X');
                    string bid = $"{date:yyyyMMdd}-{nPart}-{InventoryDb.Count + 1:D3}";

                    double pot = SafeRequestDouble("Pot Volume (L)");
                    int qty = SafeRequestInt("Quantity");
                    double cost = SafeRequestDouble("Unit Cost (Original Currency)");
                    Currency curr = SelectCurrency();
                    double rate = await GetExchangeRate(curr);

                    // Requirement: Review ALL information before saving
                    Console.Clear();
                    Console.WriteLine("--- REVIEW NEW RECORD ---");
                    Console.WriteLine($"Batch ID:      {bid}");
                    Console.WriteLine($"Plant/Species: {selectedUid}");
                    Console.WriteLine($"Date:          {date:yyyy-MM-dd}");
                    Console.WriteLine($"Nursery:       {nursery}");
                    Console.WriteLine($"Pot Volume:    {pot}L");
                    Console.WriteLine($"Quantity:      {qty} units");
                    Console.WriteLine($"Unit Cost:     {cost} {curr}");
                    Console.WriteLine($"Exchange Rate: {rate:N4} (to SEK)");
                    Console.WriteLine($"Total Cost:    {(cost * qty * rate):N2} SEK");

                    Console.Write("\nSave record? (Y/N - 'R' to Restart): ");
                    string confirm = Console.ReadLine()?.ToUpper();
                    if (confirm == "R") continue;
                    if (confirm == "Y")
                    {
                        InventoryDb.Add(new InventoryBatch
                        {
                            BatchID = bid,
```

```
                            PlantUID = selectedUid,
                            PurchaseDate = date,
                            Nursery = nursery,
                            PotVolume = pot,
                            Quantity = qty,
                            UnitCost = cost,
                            Curr = curr,
                            ExchangeRateToSek = rate,
                            UpdateReason = "Initial Entry",
                            LastUpdatedDate = DateTime.Now
                        });
                        PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                        Console.WriteLine("Saved."); Console.ReadKey();
                    }
                    break;
                }
                catch (Exception ex) when (ex.Message == "GoBack") { return; }
                catch (Exception ex) when (ex.Message == "Restart") { continue; }
            }
        }

        static async Task UpdateInventoryFlow()
        {
            var enriched = GetEnrichedInventory();
            ViewInventory(); // Selection menu includes all fields
            string input = RequestInput("\nSelect Number to Update (or B to go back)", true);
            if (input.ToUpper() == "B") return;
            if (!int.TryParse(input, out int idx) || idx < 1 || idx > enriched.Count) return;

            var b = InventoryDb.First(x => x.BatchID == enriched[idx - 1].inv.BatchID);
            string oldBid = b.BatchID;

            Console.WriteLine("\n--- UPDATING FIELDS (Press [Enter] to keep current) ---");
            // Requirement: Clarify YYYYMMDD in prompt
            string dIn = RequestInput($"Date ({b.PurchaseDate:yyyy-MM-dd}) [Use YYYYMMDD or YYYY-MM-DD]", false);
            if (!string.IsNullOrEmpty(dIn)) b.PurchaseDate = SafeParseDate(dIn);

            string nIn = RequestInput($"Nursery ({b.Nursery})", false);
            if (!string.IsNullOrEmpty(nIn)) b.Nursery = nIn;

```

```
            if (!string.IsNullOrEmpty(dIn) || !string.IsNullOrEmpty(nIn))
            {
                string nPart = b.Nursery.Length >= 3 ? b.Nursery.Substring (0, 3).ToUpper() : b.Nursery.ToUpper().PadRight(3, 'X');
                b.BatchID = $"{b.PurchaseDate:yyyyMMdd}-{nPart}-{oldBid.Split('-').Last()}";
            }

            string potIn = RequestInput($"Pot Volume ({b.PotVolume}L)", false);
            if (!string.IsNullOrEmpty(potIn)) b.PotVolume = double.Parse (potIn);

            string qtyIn = RequestInput($"Quantity ({b.Quantity})", false);
            if (!string.IsNullOrEmpty(qtyIn)) b.Quantity = int.Parse (qtyIn);

            string costIn = RequestInput($"Unit Cost ({b.UnitCost} {b.Curr})", false);
            if (!string.IsNullOrEmpty(costIn)) b.UnitCost = double.Parse (costIn);

            string cIn = RequestInput($"Change Currency? ({b.Curr})", false);
            if (!string.IsNullOrEmpty(cIn) && Enum.TryParse(cIn.ToUpper(), out Currency newCurr))
            {
                b.Curr = newCurr;
                b.ExchangeRateToSek = await GetExchangeRate(b.Curr);
            }

            b.UpdateReason = RequestInput("Reason for update", true);
            b.LastUpdatedDate = DateTime.Now;

            Console.Clear();
            Console.WriteLine("--- REVIEW UPDATED RECORD ---");
            DisplayBatchDetails(b);
            Console.Write("\nApply changes? (Y/N): ");
            if (Console.ReadLine()?.ToUpper() == "Y")
            {
                PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                Console.WriteLine("Database updated.");
            }
            else { LoadAllData(); Console.WriteLine("Cancelled."); }
            Console.ReadKey();
        }

```

```
        static void DeleteInventoryFlow()
        {
            var enriched = GetEnrichedInventory();
            ViewInventory(); // Selection menu includes all fields
            string input = RequestInput("\nEnter Number to Delete (or B to cancel)", true);
            if (input.ToUpper() == "B") throw new Exception("GoBack");
            if (int.TryParse(input, out int idx) && idx > 0 && idx <= enriched.Count)
            {
                var b = enriched[idx - 1].inv;
                Console.Clear();
                Console.WriteLine("--- CONFIRM DELETION ---");
                DisplayBatchDetails(b);
                Console.Write($"\nDelete {b.BatchID}? (Y/N): ");
                if (Console.ReadLine()?.ToUpper() == "Y")
                {
                    InventoryDb.RemoveAll(x => x.BatchID == b.BatchID);
                    PersistenceManager.SaveToJson(InventoryDb, "inventory.json");
                }
            }
        }

        static void ViewInventorySummary()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- 2. INVENTORY SUMMARY VIEW ---");
                Console.WriteLine("1. Group by Nursery | 2. Group by Date | 3. Group by Species | B. Back");

                string mode = Console.ReadLine()?.ToUpper();
                // FIX: Throws exception to stay within the "Track Inventory" menu stack
                if (mode == "B") throw new Exception("GoBack");

                var enriched = GetEnrichedInventory();
                var groups = mode switch
                {
                    "1" => enriched.GroupBy(b => b.inv.Nursery).OrderBy(x => x.Key)
                                   .Select(g => new { Header = $"NURSERY: {g.Key.ToUpper()}", Items = g }),
                    "2" => enriched.GroupBy(b => b.inv.PurchaseDate.ToString("yyyy-MM")).OrderBy(x => x.Key)
                                   .Select(g => new { Header = $"PERIOD:
```

```
                                        {g.Key}", Items = g }),
                    "3" => enriched.GroupBy(b => b.SpeciesName).OrderBy(x => x.Key)
                                    .Select(g => new { Header = $"SPECIES: {g.Key}", Items = g }),
                    _ => null
                };

                if (groups != null)
                {
                    // Prepare Export Content String (Functional Requirement)
                    string exportContent = $"INVENTORY SUMMARY - {DateTime.Now:yyyy-MM-dd HH:mm}\n" + new string('=', 115) + "\n";

                    foreach (var g in groups)
                    {
                        string headerLine = $"\n{g.Header} | Group Total: {g.Items.Sum(x => x.inv.TotalCostSek):N2} SEK";
                        Console.WriteLine(headerLine);
                        // Aligned Column Header for the sub-items
                        Console.WriteLine("{0,-20} | {1,-25} | {2,-6} | {3,-6} | {4,-15} | {5,-15}",
                            "BATCH ID", "SPECIES", "POT", "QTY", "UNIT COST", "TOTAL SEK");
                        Console.WriteLine(new string('-', 115));

                        exportContent += headerLine + "\n" + new string('-', 115) + "\n";

                        foreach (var item in g.Items.OrderBy(x => x.inv.BatchID))
                        {
                            // Aligned data row using fixed-width interpolation
                            string line = string.Format("{0,-20} | {1,-25} | {2,5}L | {3,6} | {4,10:N2} {5,-3} | {6,11:N2} SEK",
                                item.inv.BatchID,
                                item.SpeciesName.Length > 25 ? item.SpeciesName.Substring(0, 22) + "..." : item.SpeciesName,
                                item.inv.PotVolume,
                                item.inv.Quantity,
                                item.inv.UnitCost, item.inv.Curr,
                                item.inv.TotalCostSek);

                            Console.WriteLine(line);
```

```
                                exportContent += line + "\n";
                            }
                        }

                        Console.WriteLine("\nEXPORT: 1. Text | 2. Excel | 3. PDF | B. Return");
                        string choice = Console.ReadLine();
                        if (choice == "1") PersistenceManager.ExportSummary(exportContent, "Inventory_Summary");
                        else if (choice == "2") PersistenceManager.ExportToExcel(InventoryDb, "Inventory_Summary");
                        else if (choice == "3") PersistenceManager.GeneratePLReport(InventoryDb, GrowthDb, ExpenseDb);

                        if (choice == "1" || choice == "2" || choice == "3") { Console.WriteLine("Press any key..."); Console.ReadKey(); }
                    }
                }
            }

            static void ViewInventory()
            {
                Console.Clear();
                var enriched = GetEnrichedInventory();

                // Header with aligned widths (Total: 115 characters)
                // Adjusting widths to ensure separators (|) line up perfectly
                Console.WriteLine("{0,-4} | {1,-18} | {2,-25} | {3,-10} | {4,-5} | {5,-4} | {6,-12} | {7,-12}",
                    "Num", "BATCH ID", "SPECIES NAME", "DATE", "POT", "QTY", "UNIT COST", "TOTAL SEK");

                Console.WriteLine(new string('-', 115));

                for (int i = 0; i < enriched.Count; i++)
                {
                    var b = enriched[i].inv;
                    // matching the widths from the header above exactly
                    Console.WriteLine("{0,-4} | {1,-18} | {2,-25} | {3,-10:yyyy-MM-dd} | {4,-5} | {5,-4} | {6,-12} | {7,-12:N2}",
                        i + 1,
                        b.BatchID,
                        enriched[i].SpeciesName.Length > 25 ? enriched[i].SpeciesName.Substring(0, 22) + "..." : enriched[i].SpeciesName,
```

```
                    b.PurchaseDate,
                    b.PotVolume + "L",
                    b.Quantity,
                    b.UnitCost + " " + b.Curr,
                    b.TotalCostSek);
            }

            Console.WriteLine(new string('-', 115));
            //Console.WriteLine("{0,100} | Total: {1:N2} SEK", "", InventoryDb.Sum(x => x.TotalCostSek));
            // ALIGNMENT FIX:
            // The columns before 'TOTAL SEK' sum up to 101 characters (including separators).
            // Using {0,101} places the '|' exactly under the previous separators.
            // {1,12:N2} matches the {7,-12:N2} width used in the rows above.

            double grandTotal = InventoryDb.Sum(x => x.TotalCostSek);
            Console.WriteLine("{0,96} | {1,8:N2} SEK", "GRAND TOTAL:", grandTotal);
            //Console.WriteLine("{0,-86} | {1,12:N2} SEK", "GRAND TOTAL:", grandTotal);
        }

        #endregion

        #region Helpers & Persistence
        private static List<EnrichedInventoryItem> GetEnrichedInventory()
        {
            return InventoryDb.Join(SpeciesDb, inv => inv.PlantUID, spec => spec.PlantUID, (inv, spec) => new EnrichedInventoryItem
            {
                inv = inv,
                SpeciesName = $"{spec.Genus} {spec.Species} {(string.IsNullOrEmpty(spec.Cultivar) ? "" : $"\"{spec.Cultivar}\"")}"
            }).ToList();
        }

        private static string SelectPlantUID()
        {
            while (true)
            {
                Console.Clear();
                Console.WriteLine("--- SELECT BIOLOGICAL PROFILE ---");
                Console.WriteLine("{0,-4} | {1,-12} | {2,-42} | {3,-6}", "Num", "UID", "Scientific Name \"Cultivar\"", "Hardy");
                for (int i = 0; i < SpeciesDb.Count; i++)
```

```
                {
                    var s = SpeciesDb[i];
                    Console.WriteLine("{0,-4} | {1,-12} | {2,-42} | {3,-6}", i + 1, s.PlantUID, $"{s.Genus} {s.Species} \"{s.Cultivar}\"", s.HardinessLow + "C");
                }
                string input = RequestInput("\nSelect Number (R to Restart | B for Back)", true).ToUpper();
                if (input == "B") throw new Exception("GoBack");
                if (input == "R") throw new Exception("Restart");
                if (int.TryParse(input, out int idx) && idx > 0 && idx <= SpeciesDb.Count) return SpeciesDb[idx - 1].PlantUID;
                Console.WriteLine("Invalid selection. Press any key to retry..."); Console.ReadKey();
            }
        }

        private static async Task<double> GetExchangeRate(Currency target)
        {
            if (target == Currency.SEK) return 1.0;
            double rate = 1.0;
            try
            {
                string url = $"https://exchangerate-api.com{target}";
                var response = await _httpClient.GetStringAsync(url);
                using var json = JsonDocument.Parse(response);
                rate = json.RootElement.GetProperty("rates").GetProperty("SEK").GetDouble();
                Console.WriteLine($"[API] Fetched rate: 1 {target} = {rate:N4} SEK");
            }
            catch { Console.WriteLine("[API Error] Could not fetch rate."); }
            return SafeRequestDouble($"Enter/Confirm rate (Current: {rate})");
        }

        private static Currency SelectCurrency() { Console.WriteLine("1. SEK | 2. EUR | 3. USD | 4. GBP"); return Console.ReadLine() switch { "2" => Currency.EUR, "3" => Currency.USD, "4" => Currency.GBP, _ => Currency.SEK }; }

        static void ManageGrowthFlow()
        {
            while (true)
            {
                Console.Clear(); Console.WriteLine("--- 3. GROWTH ---"); Console.WriteLine("1. View | 2. Add | B. Back"); string cmd = Console.ReadLine()?.ToUpper();
```

```
                if (cmd == "B") return;
                if (cmd == "1") { foreach (var l in GrowthDb) Console.WriteLine($"{l.Date:yyyy-MM} | Batch {l.BatchID} | {l.MaxHeightCurrent}m"); Console.ReadKey(); }
                else if (cmd == "2") { string bid = RequestInput("Batch ID", true); GrowthDb.Add(new PerformanceLog { LogID = Guid.NewGuid().ToString().Substring(0, 8), BatchID = bid, Date = DateTime.Now, MaxHeightCurrent = SafeRequestDouble("Height") }); PersistenceManager.SaveToJson(GrowthDb, "logs.json"); Console.WriteLine("Growth saved."); Console.ReadKey(); }
            }
        }

        static void ManageExpenseFlow()
        {
            while (true)
            {
                Console.Clear(); Console.WriteLine("--- 4. EXPENSES ---"); Console.WriteLine("1. View | 2. Add | B. Back"); string cmd = Console.ReadLine()?.ToUpper();
                if (cmd == "B") return;
                if (cmd == "1") { foreach (var e in ExpenseDb) Console.WriteLine($"{e.Date:yyyy-MM} | {e.BatchID} | {e.MaintenanceCost} SEK"); Console.ReadKey(); }
                else if (cmd == "2") { string bid = RequestInput("Batch ID (or 'Global')", true); ExpenseDb.Add(new OperationalExpense { ExpenseID = Guid.NewGuid().ToString().Substring(0, 8), BatchID = bid, Date = DateTime.Now, MaintenanceCost = SafeRequestDouble("Cost") }); PersistenceManager.SaveToJson(ExpenseDb, "expenses.json"); Console.WriteLine("Expense saved."); Console.ReadKey(); }
            }
        }

        static void DisplayBatchDetails(InventoryBatch b)
        {
            Console.WriteLine(new string('-', 45));
            Console.WriteLine($"{"ID:",-15} {b.BatchID}");
            Console.WriteLine($"{"Plant UID:",-15} {b.PlantUID}");
            Console.WriteLine($"{"Date:",-15} {b.PurchaseDate:yyyy-MM-dd}");
            Console.WriteLine($"{"Nursery:",-15} {b.Nursery}");
            Console.WriteLine($"{"Pot Volume:",-15} {b.PotVolume}L");
            Console.WriteLine($"{"Quantity:",-15} {b.Quantity}");
            Console.WriteLine($"{"Unit Cost:",-15} {b.UnitCost} {b.Curr}");
            Console.WriteLine($"{"Reason:",-15} {b.UpdateReason}");
```

```
            Console.WriteLine(new string('-', 45));
        }

        private static DateTime SafeParseDate(string input) { string[] formats = { "yyyy-MM-dd", "yyyyMMdd" }; if (DateTime.TryParseExact(input, formats, CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime result)) return result; Console.WriteLine("Invalid format."); return DateTime.MinValue; }
        static void RestoreFlow() { var backups = PersistenceManager.GetAvailableBackups(); for (int i = 0; i < backups.Count; i++) Console.WriteLine($"{i + 1}. {backups[i].Name}"); int idx = SafeRequestInt("Select #"); PersistenceManager.RestoreBackup(backups[idx - 1].FullName, "inventory.json"); LoadAllData(); }
        static string RequestInput(string p, bool r) { Console.Write($"{p}: "); string i = Console.ReadLine(); if (i?.ToUpper() == "B") throw new Exception("GoBack"); return i; }
        static double SafeRequestDouble(string p) { while (true) if (double.TryParse(RequestInput(p, true), out double d)) return d; }
        static int SafeRequestInt(string p) { while (true) if (int.TryParse(RequestInput(p, true), out int i)) return i; }
        static DateTime SafeRequestDate(string p) { string[] f = { "yyyy-MM-dd", "yyyyMMdd" }; while (true) if (DateTime.TryParseExact(RequestInput(p, true), f, CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime res)) return res; }
        static void ShowActiveAlerts() { if (DateTime.Now.Month >= 5 && DateTime.Now.Month <= 7) Console.WriteLine("!!! ALERT: Peak Harvest !!!"); }
        static void SeedSampleData() { SpeciesDb = SeedDataGenerator.GetResearchedSpecies(); InventoryDb = SeedDataGenerator.GetSampleInventoryBatches(); PerformAutoSave(); }
        #endregion
    }
}
```

```
using BambooEnterprise.Models;
using DocumentFormat.OpenXml.Drawing;
using DocumentFormat.OpenXml.Math;
using DocumentFormat.OpenXml.Office2013.Word;
using DocumentFormat.OpenXml.Spreadsheet;
using DocumentFormat.OpenXml.Wordprocessing;
using iText.StyledXmlParser.Jsoup.Nodes;
using iText.StyledXmlParser.Jsoup.Select;
using Org.BouncyCastle.Ocsp;
using System;
using System.Collections.Generic;
using System.Diagnostics.Contracts;
using System.Runtime.InteropServices;
using static Org.BouncyCastle.Asn1.Cmp.Challenge;
namespace BambooEnterprise.Testing
{
    public static class SeedDataGenerator
    {
        public static List<InventoryBatch> GetSampleInventoryBatches()
        {
            return new List<InventoryBatch>
            {
                new InventoryBatch {
                    BatchID = "20220611-KIM-001",
                    PlantUID = "PHY-VIV-AUR",
                    PurchaseDate = new DateTime(2022, 06, 11),
                    Nursery = "Kimmei",
                    PotVolume = 10,
                    Quantity = 2,
                    UnitCost = 45.00,
                    Curr = Currency.EUR,
                    ExchangeRateToSek = 11.20,
                    UpdateReason = "Initial Seed Data",
                    LastUpdatedDate = DateTime.Now
                },
                new InventoryBatch {
                    BatchID = "20240315-BAM-002",
                    PlantUID = "FAR-NIT-OBE",
                    PurchaseDate = new DateTime(2024, 03, 15),
                    Nursery = "BambooGardens",
                    PotVolume = 5,
                    Quantity = 5,
                    UnitCost = 250.00,
                    Curr = Currency.SEK,
                    ExchangeRateToSek = 1.0,
                    UpdateReason = "Initial Seed Data",
                    LastUpdatedDate = DateTime.Now
                }
            };
```

```
        }
        public static List<BambooSpecies> GetResearchedSpecies()
        {
            var list = new List<BambooSpecies>();
            // Runners
            list.Add(CreateMono("PHY-VIV-HUA", "Phyllostachys", "vivax", "Huanwenzhu", 12, 10, -21, 6, "Green culm, yellow sulcus.Snöbrott risk."));
            list.Add(CreateMono("PHY-VIV-MCC", "Phyllostachys", "vivax", "McClure", 14, 12, -21, 6, "Classic green timber. Large leaves."));
            list.Add(CreateMono("PHY-VIV-AUR", "Phyllostachys", "vivax", "Aureocaulis", 12, 11, -21, 6, "Yellow culm, green stripes. High aesthetics."));
            list.Add(CreateMono("PHY-PAR-NOM", "Phyllostachys", "parvifolia", "", 10, 9, -25, 5, "Extremely hardy timber. Edible shoots."));
            list.Add(CreateMono("PHY-ATR-INC", "Phyllostachys","atrovaginata", "", 8, 7, -23, 5, "Air canals for wet soil. Scented."));
            list.Add(CreateMono("PHY-AUR-NOM", "Phyllostachys", "aureosulcata", "", 7, 4, -25, 5, "Green culm, yellow sulcus.Zig - zag bases."));
            list.Add(CreateMono("PHY-AUR-AUR", "Phyllostachys", "aureosulcata", "Aureocaulis", 7, 4, -25, 5, "Solid yellow. Very hardy."));
            list.Add(CreateMono("PHY-AUR-SPE", "Phyllostachys", "aureosulcata", "Spectabilis", 7, 4, -25, 5, "Yellow culm, green sulcus.Red shoots."));
            list.Add(CreateMono("PHY-AUR-ARG", "Phyllostachys", "aureosulcata", "Argus", 7, 4, -25, 5, "Green stripes on yellow."));
            list.Add(CreateMono("PHY-NIG-NOM", "Phyllostachys", "nigra", "", 5, 3, -18, 5, "Jet black culms. Needs shelter."));
            list.Add(CreateMono("PHY-NIG-HEN", "Phyllostachys", "nigra", "Henonis", 10, 8, -23, 6, "Giant grey timber. Very hardy."));
            list.Add(CreateMono("PHY-NIG-PUN", "Phyllostachys", "nigra", "Punctata", 7, 4, -20, 6, "Speckled black culms."));
            list.Add(CreateMono("PHY-BIS-NOM", "Phyllostachys", "bissetii", "", 7, 4, -26, 4, "Best Swedish windbreak."));
            list.Add(CreateMono("PHY-DEC-NOM", "Phyllostachys", "decora", "", 6, 4, -22, 5, "Drought tolerant runner."));
            list.Add(CreateMono("PHY-NUD-NOM", "Phyllostachys", "nuda", "", 6, 4, -26, 4, "Early purple / black nodes."));
            list.Add(CreateMono("SAS-KUR-NOM", "Sasa", "kurilensis", "", 2, 1.5, -30, 6, "Northernmost species. Large leaves."));
            list.Add(CreateMono("BAS-QIN-NOM", "Bashania","qingchengshanensis", "", 4, 2, -20, 4, "Strong,
```

```
                stiff culms."));
            list.Add(CreateMono("SEM-FAS-NOM", "Semiarundinaria", "fastuosa", "", 8, 4, -20, 6, "Upright columnar habit."));
            // Clumpers
            list.Add(CreateSym("FAR-NIT-OBE", "Fargesia", "nitida x murieliae", "Obelisk", 4, -25, 5, "Tallest upright columnar hedge."));
            list.Add(CreateSym("FAR-MUR-RZB", "Fargesia", "murieliae", "Red Zebra", 3, -23, 5, "Dark red / green contrast stems."));
            list.Add(CreateSym("FAR-NIT-SCH", "Fargesia", "nitida x murieliae", "Schensbossen", 4, -26, 5, "Exceptional hardiness."));
            list.Add(CreateSym("FAR-SPP-RUF", "Fargesia", "sp.", "Rufa", 2.5, -23, 4, "Non-curling leaves. Early shoots."));
            list.Add(CreateSym("FAR-ROB-CAM", "Fargesia", "robusta","Campbell", 5, -20, 4, "Earliest shoots. White pattern."));
            list.Add(CreateSym("FAR-NIT-JUR", "Fargesia", "nitida", "Jürgen", 3.5, -26, 5, "Upright, dark culms."));
            list.Add(CreateSym("FAR-JIU-ONE", "Fargesia", "sp. Jiuzhaigou", "1", 3, -25, 5, "Red Panda.Cherry stems in sun."));
            list.Add(CreateSym("FAR-JIU-DPU", "Fargesia", "sp. Jiuzhaigou", "Deep Purple", 3, -25, 5, "Deepest purple culm color."));
            list.Add(CreateSym("FAR-DEM-GER", "Fargesia", "demissa", "Gerry", 3, -25, 5, "Almost black culms."));
            list.Add(CreateSym("FAR-DEN-XI2", "Fargesia", "denudata", "Xian 2", 4, -22, 5, "Cloud - like elegant habit."));
            list.Add(CreateSym("FAR-SPP-KR5", "Fargesia", "sp.", "KR5287", 4, -20, 6, "Rare collector blueish culms."));
return list;
        }
        private static Monopodial CreateMono(string uid, string g, string
        s, string c, double h, double d, double hard, int shoot, string
        note)
        => new Monopodial
        {
            PlantUID = uid,
            Genus = g,
            Species = s,
            Cultivar = c,
            Habit = GrowthHabit.Monopodial,
            HeightHigh =
        h,
            DiameterHigh = d,
            HardinessLow = hard,
            TypicalShootingMonth = shoot,
```

```
            SverigeSpecifikt = note,
            ShootEdibilityRank = 4,
            HeightLow = h * 0.7,
            DiameterLow = d * 0.6
        };
        private static Sympodial CreateSym(string uid, string g, string s,
        string c, double h, double hard, int shoot, string note)
        => new Sympodial
        {
            PlantUID = uid,
            Genus = g,
            Species = s,
            Cultivar = c,
            Habit = GrowthHabit.Sympodial,
            HeightHigh = h,
            HardinessLow = hard,
            TypicalShootingMonth = shoot,
            SverigeSpecifikt = note,
            ClumpSpacing = 1.0,
            HeightLow = h * 0.7
        };
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Text.Json.Serialization;

namespace BambooEnterprise.Models
{
    public enum GrowthHabit { Monopodial, Sympodial }

    // Required for the ExchangeRate-API integration
    public enum Currency { SEK, EUR, USD, GBP }

    [JsonDerivedType(typeof(Monopodial), typeDiscriminator: "monopodial")]
    [JsonDerivedType(typeof(Sympodial), typeDiscriminator: "sympodial")]
    public abstract class BambooSpecies
    {
        public string PlantUID { get; set; }
        public string Genus { get; set; }
        public string Species { get; set; }
        public string Cultivar { get; set; }
        public GrowthHabit Habit { get; set; }
        public double HeightLow { get; set; }
        public double HeightHigh { get; set; }
        public double DiameterLow { get; set; }
        public double DiameterHigh { get; set; }
        public double HardinessLow { get; set; }
        public int TypicalShootingMonth { get; set; }
        public string SverigeSpecifikt { get; set; }
        public int ShootEdibilityRank { get; set; }
        public int TimberQualityRank { get; set; }
    }

    public class Monopodial : BambooSpecies { public double RhizomeDepth { get; set; } }
    public class Sympodial : BambooSpecies { public double ClumpSpacing { get; set; } }

    public class InventoryBatch
    {
        public string BatchID { get; set; }
        public string PlantUID { get; set; }
        public DateTime PurchaseDate { get; set; }
        public string Nursery { get; set; }
        public double PotVolume { get; set; }
        public int Quantity { get; set; }
        public double UnitCost { get; set; }
        public Currency Curr { get; set; }
        public double ExchangeRateToSek { get; set; }

        // Calculated Property
```

```
        public double TotalCostSek => (UnitCost * Quantity) * ExchangeRateToSek;

        // Audit Trail Fields
        public string UpdateReason { get; set; }
        public DateTime LastUpdatedDate { get; set; }

        public bool IsPeakCulmAge(int currentYear) => (currentYear - PurchaseDate.Year) >= 4;
    }

    public class PerformanceLog
    {
        public string LogID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public int NewShootCount { get; set; }
        public double MaxHeightCurrent { get; set; }
        public double CulmRevenueSek { get; set; }
        public double ShootRevenueSek { get; set; }
    }

    public class OperationalExpense
    {
        public string ExpenseID { get; set; }
        public string BatchID { get; set; }
        public DateTime Date { get; set; }
        public double MaintenanceCost { get; set; }
        public double EquipmentCost { get; set; }
        public double LaborCost { get; set; }
        public double TransportCost { get; set; }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.Json;
using BambooEnterprise.Models;
using ClosedXML.Excel;
using iText.IO.Font.Constants;
using iText.Kernel.Font;
using iText.Kernel.Pdf;
using iText.Layout;
using iText.Layout.Element;
//using iText.Layout.Element.Paragraph;
using iText.Layout.Properties;

namespace BambooEnterprise.Data
{
    public static class PersistenceManager
    {
        private static readonly string Root = AppDomain.CurrentDomain.BaseDirectory;
        private static readonly string DataDir = Path.Combine(Root, "Data");
        private static readonly string BackupDir = Path.Combine(DataDir, "Backups");
        private static readonly string PdfDir = Path.Combine(Root, "Reports", "PDF");
        private static readonly string ExcelDir = Path.Combine(Root, "Reports", "Excel");
        private static readonly string SummaryDir = Path.Combine(Root, "Reports", "Summary");

        static PersistenceManager()
        {
            Directory.CreateDirectory(DataDir);
            Directory.CreateDirectory(BackupDir);
            Directory.CreateDirectory(PdfDir);
            Directory.CreateDirectory(ExcelDir);
            Directory.CreateDirectory(SummaryDir);
        }

        private static void CreateBackup(string fileName)
        {
            try
            {
                string source = Path.Combine(DataDir, fileName);
                if (File.Exists(source))
                {
                    string bName = $"{Path.GetFileNameWithoutExtension
```

```
                        (fileName)}_{DateTime.Now:yyyyMMdd_HHmmss}.json";
                    string bPath = Path.Combine(BackupDir, bName);
                    File.Copy(source, bPath, true);
                    Console.WriteLine($"[BACKUP] Created: {bPath}");
                    var old = new DirectoryInfo(BackupDir).GetFiles
                      ("*.json").OrderByDescending(f =>
                      f.CreationTime).Skip(10);
                    foreach (var f in old) f.Delete();
                }
            }
            catch { }
        }

        public static void SaveToJson<T>(List<T> data, string fileName)
        {
            try
            {
                CreateBackup(fileName);
                string path = Path.Combine(DataDir, fileName);
                File.WriteAllText(path, JsonSerializer.Serialize(data,
                  new JsonSerializerOptions { WriteIndented = true }));
                Console.WriteLine($"[SAVE] Successful: {path}");
            }
            catch (Exception ex) { throw new Exception($"Save Failed:
              {ex.Message}"); }
        }

        public static List<T> LoadFromJson<T>(string fileName)
        {
            string path = Path.Combine(DataDir, fileName);
            if (!File.Exists(path)) return new List<T>();
            try
            {
                Console.WriteLine($"[LOAD] Reading: {path}");
                return JsonSerializer.Deserialize<List<T>>
                  (File.ReadAllText(path)) ?? new List<T>();
            }
            catch { return new List<T>(); }
        }

        public static List<FileInfo> GetAvailableBackups() => new
          DirectoryInfo(BackupDir).GetFiles("*.json").OrderByDescending(f
          => f.CreationTime).ToList();

        public static void RestoreBackup(string source, string target)
        {
            string targetPath = Path.Combine(DataDir, target);
            CreateBackup(target);
            File.Copy(source, targetPath, true);
```

```
            Console.WriteLine($"[RESTORE] Overwritten: {targetPath}");
        }

        public static void ExportSummary(string content, string name)
        {
            string path = Path.Combine(SummaryDir, $"{name}_{DateTime.Now:yyyyMMdd}.txt");
            File.WriteAllText(path, content);
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }

        public static void ExportToExcel<T>(List<T> data, string name)
        {
            string path = Path.Combine(ExcelDir, $"{name}.xlsx");
            using (var wb = new XLWorkbook()) { wb.Worksheets.Add("Report").Cell(1, 1).InsertTable(data); wb.SaveAs(path); }
            Console.WriteLine($"[EXPORT] Saved: {path}");
        }

        public static void GeneratePLReport(List<InventoryBatch> batches, List<PerformanceLog> logs, List<OperationalExpense> expenses)
        {
            string path = Path.Combine(PdfDir, "Bamboo_PL_Report.pdf");
            using (var writer = new PdfWriter(path))
            using (var pdf = new PdfDocument(writer))
            using (var doc = new Document(pdf))
            {
                doc.Add(new Paragraph("Financial Audit & P&L Report").SetFontSize(18).SetFont(PdfFontFactory.CreateFont(StandardFonts.HELVETICA_BOLD)));
                doc.Add(new Paragraph($"Generated: {DateTime.Now:yyyy-MM-dd HH:mm}"));

                // Expanded to 7 columns to include Original Cost and Currency
                Table table = new Table(7).SetWidth(UnitValue.CreatePercentValue(100));
                table.AddHeaderCell("Batch ID");
                table.AddHeaderCell("Orig. Cost");
                table.AddHeaderCell("Curr");
                table.AddHeaderCell("Total SEK");
                table.AddHeaderCell("OpEx SEK");
                table.AddHeaderCell("Net SEK");
                table.AddHeaderCell("Audit Trail (Reason)");

                foreach (var b in batches)
                {
                    double rev = logs.Where(l => l.BatchID ==
```

```
                        b.BatchID).Sum(l => l.CulmRevenueSek +
                        l.ShootRevenueSek);
                    double opex = expenses.Where(e => e.BatchID ==
                        b.BatchID || e.BatchID == "Global").Sum(e =>
                        e.MaintenanceCost);
                    double totalCost = b.TotalCostSek + opex;
                    double net = rev - totalCost;

                    table.AddCell(b.BatchID);
                    table.AddCell(b.UnitCost.ToString("N2"));
                    table.AddCell(b.Curr.ToString());
                    table.AddCell(b.TotalCostSek.ToString("N2"));
                    table.AddCell(opex.ToString("N2"));
                    table.AddCell(net.ToString("N2"));
                    table.AddCell($"{b.UpdateReason}
                        ({b.LastUpdatedDate:yyyy-MM-dd})");
                }
                doc.Add(table);
            }
            Console.WriteLine($"[EXPORT] Saved to PDF: {path}");
        }
    }
}
```